

Speech Processing and Multimodal AI

Mini-Project

Mini-project ideas and an indicative grading rubric

Note: none of the mini-project ideas below require training from scratch, no large-scale fine-tuning, but some could be fine-tuned with LoRA on medium sized datasets that can be done with the available GPU credits that teams have access to. Teams should select a preferred option and a backup too (in case another team gets their first choice).

Teams can do many things, e.g. swap in a newer model, use the dataset differently, or a different dataset, or combine an LLM or a different LLM. You are encouraged to combine this weeks' themes (speech, non-music audio, or image+text diffusion) with an LLM where it adds value, or with concepts learned in previous weeks. You don't need to follow the suggestions below, you can propose your own ideas. Note that 7 of the project ideas were generated by AI and modified by me (I'll leave you to guess which).

Project ideas

Idea 1: Few-Shot Voice Cloning Narrator

Clone a target voice from only a few minutes of reference audio and use it to narrate LLM-generated content (e.g. personalised audiobook chapters or study-note read-alouds). You can find online tools to do this, but you are going to build your own.

Suggested starting models / data: Coqui XTTS-v2 or FunAudioLLM CosyVoice3-0.5B (both support few-shot/zero-shot cloning); 3–10 minutes of a team member's voice as the reference, but you must get their permission first.

What the team builds (~10 person-hours): Fine-tune/adapt the cloning model on the reference voice; chain an LLM to convert source text (notes, articles) into narration-ready script; produce a short narrated demo clip live.

Course concepts demonstrated: Non-ASR speech synthesis, few-shot adaptation, LLM content transformation, ethical-use discussion (consent, disclosure of synthetic voice).

Idea 2: Accent / Prosody Softening for Support Calls

Convert or reshape speech prosody (not words) to make recorded customer-support snippets sound calmer/clearer, without touching transcription — a voice-to-voice task.

Suggested starting models / data: A voice-conversion or prosody-transfer model (e.g. an open VC model such as FreeVC/kNN-VC, or fine-tuned XTTS-v2 as a voice-conversion target) on small VCTK speaker subsets.

What the team builds (~10 person-hours): Fine-tune the conversion model on a handful of speaker pairs; build a before/after demo playing original vs. converted clips; quantify change with a simple prosody metric (pitch variance, speaking rate).

Course concepts demonstrated: Non-ASR speech-to-speech conversion, prosody analysis, small-data fine-tuning.

Idea 3: Speech Emotion Coach

A practice tool (call-centre training, public-speaking rehearsal) that classifies the emotional tone/energy of a speaker's recording and has an LLM turn that into constructive coaching feedback.

Suggested starting models / data: wav2vec2/HuBERT-based emotion classifier (e.g. wav2vec2-emotion) fine-tuned on a small RAVDESS or CREMA-D subset.

What the team builds (~10 person-hours): Fine-tune the classifier head on the chosen subset; feed per-utterance predictions (valence/arousal or category) into an LLM prompt that produces a short coaching note.

Course concepts demonstrated: Non-ASR paralinguistic speech classification, you can use an LLM-as-feedback-generator, evaluation of a small fine-tune.

Idea 4: Acoustic Safety / Anomaly Monitor

A monitor that listens for specific safety-relevant non-speech, non-music sounds (glass breaking, alarms, machinery faults, smoke-alarm beeps) and has an LLM draft a natural-language alert.

Suggested starting models / data: Audio Spectrogram Transformer (MIT/ast-finetuned-audioset) or PANNs as backbone; fine-tune on a small labelled subset of ESC-50 or UrbanSound8K restricted to the safety-relevant classes.

What the team builds (~10 person-hours): Fine-tune the classifier on the chosen classes; stream short clips through it live; on detection, prompt an LLM to generate a plain-language alert message; show a short 'incident' demo reel.

Course concepts demonstrated: Non-music audio event classification, small-data fine-tuning, LLM-generated natural-language summarisation of model output.

Idea 5: Bioacoustic Species Spotter

Classify wildlife calls (birds, frogs, insects) from a small labelled subset of a public bioacoustic dataset, then auto-generate a short ecological field-note style report.

Suggested starting models / data: AST or a lightweight CNN audio classifier fine-tuned on a small subset of BirdCLEF or a curated Xeno-canto sample (a handful of species, capped clip count). Also look at BirdNet, don't copy it though, enhance it to do something it can't do currently (e.g. some Singapore-specific birds which it doesn't currently recognise - something NParks told me about).

What the team builds (~10 person-hours): Fine-tune on the chosen species subset; run inference over a small held-out set (important that you test on different recordings to what you train with); pipe predictions + confidence into an LLM to draft a one-paragraph survey note per clip; including accuracy and example notes.

Course concepts demonstrated: Non-music bioacoustic classification, dataset curation under time constraints, LLM-assisted report generation.

Idea 6: Text-to-Sound-Effect Generator

Generate short, non-musical sound effects (footsteps, doors, sci-fi UI blips, weather) from text prompts for game/video prototyping, fine-tuned toward one niche sound domain. This one is fun!!

Suggested starting models / data: Use AudioLDM / AudioLDM2 (text-to-audio diffusion) as base; get a small curated subset of a sound-effects library or Clotho/AudioCaps filtered to non-music clips for the target domain.

What the team builds (~10 person-hours): LoRA or lightweight fine-tune the diffusion model with the new samples; build an LLM prompt-expander that turns short requests ('a heavy door creaking') into richer generation prompts; demo a small sound-pack generated live.

Course concepts demonstrated: Non-music audio diffusion, domain-specific fine-tuning, LLM prompt engineering for generative audio.

Idea 7: Image Generator from Audio

Fine-tune an image diffusion model with LoRA on the outputs from an audio recogniser from a niche database of labelled audio and images. Using the text prompts is easy, fine tuning with the audio, you might need to feed the audio embeddings in as well (look at CLAP).

Possible starting models / data: FLUX.1-schnell, Tongyi Z-Image-Turbo, or SDXL as base; a small (30–150 image) curated dataset for the target domain.

What the team builds (~10 person-hours): Train a LoRA adapter on the curated set; build an LLM prompt-assistant that expands audio into short briefs and then into well-formed generation prompts; demo comparing base model vs. fine-tuned output.

Course concepts demonstrated: Multimodal image diffusion, parameter-efficient fine-tuning (LoRA), LLM-assisted prompting, sound event classification.

Idea 8: Instruction-Based Image Editing Assistant

An assistant that takes a natural-language instruction ("remove the background hum," "make it sound like it's in a large hall," "isolate the dog bark" or "add a car horn") and applies it to an uploaded non-music audio clip via diffusion-based editing.

Suggested starting models / data: AUDIT (instruction-guided latent diffusion audio editor) or AudioLDM2 adapted for edit-style conditioning as the base; a small subset of an audio-editing pair dataset (e.g. a curated slice of AudioCaps/Clotho with synthetic before/after edit pairs — add reverb, denoise, insert/remove a sound event) restricted to non-music clips. Look here: <https://github.com/open-mmlab/Amphion/blob/main/egs/tta/RECIPE.md>

What the team builds (~10 person-hours): Fine-tune/adapt the diffusion editor on the small instruction-pair subset; use an LLM to normalise loose user requests ("get rid of that background noise") into the structured edit commands the model expects; live-demo several before/after edits on volunteered or public clips.

Course concepts demonstrated: Non-music audio diffusion, instruction parsing via LLM, small-scale adaptation of an editing model, before/after evaluation.

Idea 9: Speech-to-tone Voice Link

A "sonic voice modem": compress a short speech clip into a compact token/latent representation, modulate that representation into an audible (or near-ultrasonic) tone burst, transmit it as sound, then reverse the chain (demodulate and resynthesize) to recover speech that sounds like the original.

Suggested starting models / data: A neural audio codec as the speech "tokenizer/vocoder" pair — Meta's EnCodec (facebook/encodec_24khz), Descript Audio Codec, or SNAC are existing methods that can turn speech into a low-bitrate discrete token stream and back. You need to feed that into an existing data-over-sound / acoustic-modem library (e.g. ggwave) to make a working reference implementation. No training data strictly required, because you can use the codec off-the-shelf, but you might want to consider an optional light fine-tuning of the decoder.

What the team builds (~10 person-hours): Pipeline = speech → codec encoder → discrete tokens (bytes) → tone modulation → play through a speaker → record via a mic → demodulate → codec decoder → reconstructed speech. The modulation stage is where most of the design work and interesting trade-offs live so it is useful to give some hints here:

- *Scheme choice:* start with multi-tone FSK (each symbol = one of N simultaneous or sequential frequencies, DTMF-style) since it's simple to demodulate with an FFT; ggwave already implements several usable modes (audible-fast, audible-normal, ultrasonic) the team can benchmark instead of building from scratch.
- *Symbol rate vs. reliability:* faster symbol rates pack more codec tokens per second but are more fragile over a real speaker→mic channel (room echo, background noise, mic frequency response) — have the team sweep 2–3 symbol rates and report the error rate at each.
- *Frequency band:* use audible tones (since they are easy to debug by ear) but in a real system you could use near-ultrasonic (~18–20 kHz, which is "silent" to most listeners, although cheap speaker/mic have rolloff at that range so you would need a good speaker). You could have an amazing demo.

- *Sync & framing*: a short chirp or known preamble tone at the start of each burst (e.g. 10101010) so the receiver can find the start of the message and correct for clock drift/playback latency. Without this strict synchronisation, the demodulation will probably fail every time.
- *Error resilience*: since token bytes are not fault-tolerant, you can add a light error-correction or checksum layer (e.g. simple parity/CRC, or repetition coding) so a few flipped tones don't corrupt the whole reconstructed utterance — and demo what happens with it turned off vs. on.
- *Security*: optionally you can encrypt at the transmitter and decrypt at the receiver.

Live demo: speak a sentence, hear the tones play and record them. Play them back into the decoder. You can try adding noise to see how robust it is.

Course concepts demonstrated: Non-ASR speech representation learning (neural codec tokens as a compact speech "alphabet"), acoustic/physical-layer signal design (modulation scheme, symbol rate, framing, error resilience), and small-scale adaptation of an existing codec for a very constrained, lossy transmission channel.

Idea 10: Your Own Idea For a Project

Pitch an idea to me with similar difficulty level, that can lead to a demo. Be imaginative, look for something interesting and fun, not simple and boring. You won't lose marks by trying – even if it doesn't work fully you can get marks from following the correct development process. Make sure it fulfils what the mini-project needs to do.

Only proceed with your idea after you have clearance, by email, from me.

Team & Scope Guidelines

- Fine-tuning / adaptation only: LoRA, adapters, prompt-tuning, or light full fine-tunes on small data slices. No pretraining from scratch, no multi-day training runs. Always adapt good existing models.
- ~10 person-hours total for the team, not per person. The teams should split into sub-groups where there are different parts to the project. e.g. split across data prep, fine-tuning, integration, and demo prep accordingly. Be careful of dependencies (where you are waiting for others) – make sure you have interim data to use in the meantime. Make good use of AI coding tools to accelerate progress.
- Data: use small public subsets or small amounts of self-collected data (get consent before recording people). Favour a few hundred examples, not full datasets. We don't have time to do anything big.
- At least one course theme (speech, non-music audio, or image/text diffusion) must be extended with something new, which could be a fine-tune, a newer model slotted in, or an LLM integration layer, or some other integration with what you have learned over the past 5 weeks.
- Make a fallback video for the demo: pre-record it working, in case of live GPU/inference hiccups during the 10-minute demo slot.

Deliverables

- A 10 minute live demo on Day 5.
- A maximum 2 page A4 summary report uploaded to the course dropbox by the end of Day 5. This should have your project title, team number, and outline:
 - exactly what your task is
 - what training data you used, what testing data, what existing model(s) you started with
 - what you did, i.e. which techniques you used, highlight anything you learned in this course
 - the results you achieved.
 - A very brief reflection (what you would change if you did it again)/further work (if you have time)

Indicative Grading Rubric — 100 Points Total

This applies to the combined 10-minute in-person team demo and the 2-page submitted PDF report, and is indicative only as elements may change based on class progress.

Criterion	Pts	Scoring guide
Technical Integration	20	27–30: Clear, correct fine-tuning/adaptation of a well-chosen base model; sound understanding of why the technique fits the task. 19–26: Fine-tuning works but choices are only partly justified, or understanding of the method is shaky. 10–18: Base model barely adapted, or technique misapplied. 0–9: No real fine-tuning/adaptation attempted; base model used as-is.
Live Demo	20	23–25: Demo runs smoothly end-to-end, output is legible/audible to the room, team narrates clearly. 16–22: Demo works with minor hiccups or unclear narration. 8–15: Demo partially works or relies heavily on pre-recorded fallback. 0–7: Demo fails to show a working result.
Creative Integration of Course Concepts	30	14–15: Thoughtfully combines two+ themes (e.g. speech/audio model + LLM, or diffusion + LLM) in a way that adds real value. 9–13: Combines themes but the added layer feels superficial. 4–8: Single theme only, minimal integration. 0–3: No meaningful integration of week's concepts.
Written Report (2-page PDF)	20	18–20: Concise, technically accurate; states model(s)/data used, fine-tuning approach, results/limitations, and ethical considerations where relevant. 13–17: Covers most of the above but missing detail or clarity in one area. 7–12: Thin on technical detail or results. 0–6: Missing, or does not describe what was actually built.
Teamwork & Process	10	9–10: Involvement of entire team, well planned. 6–8: Reasonable plan and good buy-in. 3–5: Poor planning or lack of teamwork. 0–2: No process/no team.